```
`timescale 1ns / 1ps
`default_nettype none
`include "parameters.vh"

module CSR(
    input wire i_clk,
    input wire i_rst_n,
    // Interrupts
    input wire i_external_interrupt,
    input wire i_software_interrupt,
    // Exceptions
    input wire i_is_inst_illegal,
    input wire i_is_ecall,
    input wire i_is_ebreak,
    input wire i_is_mret,
    // Load/Store Misaligned
    input wire[6:0] i_opcode, // opcode types
    input wire[4:0] i_funct5, // funct5 field for AMO operations
    input wire[31:0] i_y, // address used in load/store/jump/branch
    // CSR Instruction
    input wire[2:0] i_funct3, // CSR instruction operation
    input wire[11:0] i_csr_index, // immediate value from decoder
    input wire[31:0] i_imm, // new value for CSR (immediate type)
    input wire[31:0] i_rs1, // Source register 1 value
    output reg[31:0] o_csr_out, // CSR value to be loaded to base reg
    // Trap-Handler
    input wire[31:0] i_pc, // Program Counter
    input wire writeback_change_pc, // High if writeback will issue change_pc
(overrides CSR updates)
    output reg[31:0] o_return_address, // MEPC CSR
    output reg[31:0] o_trap_address, // MTVEC CSR
    output reg o_go_to_trap_q, // High if exception/interrupt detected
    output reg o_return_from_trap_q, // High before returning from trap (via MRET)
    input wire i_minstret_inc // Increment minstret after instruction execution
);

    // CSR Registers
    reg [31:0] MVENDORID_reg, MARCHID_reg, MIMPID_reg, MHARTID_reg;
    reg [31:0] MSTATUS_reg, MISA_reg, MIE_reg, MTVEC_reg;
    reg [31:0] MSCRATCH_reg, MEPC_reg, MCAUSE_reg, MTVAL_reg, MIP_reg;

    // Detect AMO Instruction
    wire is_amo = (i_opcode == `AMO);

    // AMO Result Register
    reg [31:0] amo_result;
```

```
    // Atomic Operation Execution
    always @* begin
        if (is_amo) begin
            case (i_funct5)
                `AMO_ADD:  amo_result = o_csr_out + i_rs1;
                `AMO_SWAP: amo_result = i_rs1;
                `AMO_XOR:  amo_result = o_csr_out ^ i_rs1;
                `AMO_AND:  amo_result = o_csr_out & i_rs1;
                `AMO_OR:   amo_result = o_csr_out | i_rs1;
                `AMO_MIN:  amo_result = ($signed(o_csr_out) < $signed(i_rs1)) ?
o_csr_out : i_rs1;
                `AMO_MAX:  amo_result = ($signed(o_csr_out) > $signed(i_rs1)) ?
o_csr_out : i_rs1;
                `AMO_MINU: amo_result = (o_csr_out < i_rs1) ? o_csr_out : i_rs1;
                `AMO_MAXU: amo_result = (o_csr_out > i_rs1) ? o_csr_out : i_rs1;
                default:  amo_result = o_csr_out;
            endcase
        end
    end

    always @(posedge i_clk or negedge i_rst_n) begin
        if (!i_rst_n) begin
            // Reset all CSR registers
            MVENDORID_reg <= 32'h0;
            MARCHID_reg <= 32'h0;
            MIMPID_reg <= 32'h0;
            MHARTID_reg <= 32'h0;
            MSTATUS_reg <= 32'h0;
            MISA_reg <= 32'h0;
            MIE_reg <= 32'h0;
            MTVEC_reg <= 32'h0;
            MSCRATCH_reg <= 32'h0;
            MEPC_reg <= 32'h0;
            MCAUSE_reg <= 32'h0;
            MTVAL_reg <= 32'h0;
            MIP_reg <= 32'h0;
            o_go_to_trap_q <= 0;
            o_return_from_trap_q <= 0;
        end else begin
            if (is_amo) begin
                // Write back the AMO result to CSR
                case (i_csr_index)
                    12'h300: MSTATUS_reg <= amo_result; // MSTATUS
                    12'h301: MISA_reg <= amo_result; // MISA
                    12'h304: MIE_reg <= amo_result; // MIE
```

```
                12'h305: MTVEC_reg <= amo_result; // MTVEC
                12'h340: MSCRATCH_reg <= amo_result; // MSCRATCH
                12'h341: MEPC_reg <= amo_result; // MEPC
                12'h342: MCAUSE_reg <= amo_result; // MCAUSE
                12'h343: MTVAL_reg <= amo_result; // MTVAL
                12'h344: MIP_reg <= amo_result; // MIP
            endcase
        end else if (i_funct3 != 3'b000) begin
            // Handle normal CSR operations
            case (i_funct3)
                3'b001: o_csr_out <= i_rs1; // CSRRW
                3'b010: o_csr_out <= o_csr_out | i_rs1; // CSRRS
                3'b011: o_csr_out <= o_csr_out & (~i_rs1); // CSRRC
                3'b101: o_csr_out <= i_imm; // CSRRWI
                3'b110: o_csr_out <= o_csr_out | i_imm; // CSRRSI
                3'b111: o_csr_out <= o_csr_out & (~i_imm); // CSRRCI
            endcase
        end

        // Handle traps
        if (o_go_to_trap_q) begin
            o_return_address <= i_pc;
            o_trap_address <= MTVEC_reg;
        end
        if (i_is_mret) begin
            o_return_from_trap_q <= 1;
        end
    end
end

endmodule
```